

6. Networkx及其基本使用

我们以前见识过各种各样的图。但是除了几何图形之外，关系图上我们有哪些数学方法可以分析呢？这一章我们利用networkx支持图论当中一些基本问题的求解。本章的内容包括：

- [6.1 利用networkx创建图](#)
- [6.2 利用networkx分析图](#)
- [6.3 利用networkx求解图的一些基本问题](#)

6.1 利用networkx创建图

6.1.1 利用networkx创建无向图

使用 networkx 库来创建和分析无向图是一个直观且强大的方法。 networkx 是一个Python包，用于创建、操作复杂网络的结构、动态和功能。以下是如何使用 networkx 来创建无向图并进行简单分析的基本步骤。

安装 networkx

如果你还没有安装 networkx ，你可以通过pip来安装它：

```
pip install networkx
```

创建无向图

在 networkx 中，你可以使用 Graph 类来创建无向图。

```
import networkx as nx

# 创建一个空的无向图
G = nx.Graph()

# 添加节点
G.add_node(1)
G.add_nodes_from([2, 3, 4])

# 添加边
G.add_edge(1, 2)
G.add_edges_from([(2, 3), (3, 4), (4, 1)])

# 查看图
print(G.nodes()) # 查看节点
print(G.edges()) # 查看边
```

简单分析

1. 图的度 (Degree)

节点的度是指与该节点连接的边的数量。在无向图中，每个连接算作一条边。

```
print(G.degree()) # 查看所有节点的度
for node, degree in G.degree():
    print(f"Node {node} has degree {degree}")
```

2. 图的邻接节点

你可以获取某个节点的所有邻接节点。

```
print(list(G.neighbors(1))) # 获取节点1的所有邻接节点
```

3. 图的连通性

检查图是否是连通的（即图中任意两个节点之间都存在路径）。

```
print(nx.is_connected(G)) # 对于无向图，检查图是否连通
```

4. 图的遍历

你可以使用深度优先搜索（DFS）或广度优先搜索（BFS）来遍历图。

```
# 深度优先搜索
print(list(nx.dfs_preorder_nodes(G, source=1)))

# 广度优先搜索
print(list(nx.bfs_edges(G, source=1)))
```

5. 图的绘图

虽然绘图不是直接的分析步骤，但它对于理解图的结构非常有帮助。

```
import matplotlib.pyplot as plt

# 使用matplotlib绘制图
nx.draw(G, with_labels=True, font_weight='bold')
plt.show()
```

6.1.2 利用networkx创建有向图

在 networkx 中，创建有向图并分析它与无向图非常相似，但你需要使用 DiGraph 类而不是 Graph 类。以下是如何使用 networkx 来创建有向图并进行简单分析的基本步骤。

创建有向图

```
import networkx as nx

# 创建一个空的有向图
DG = nx.DiGraph()

# 添加节点
DG.add_node(1)
DG.add_nodes_from([2, 3, 4])

# 添加有向边
DG.add_edge(1, 2)
DG.add_edges_from([(2, 3), (3, 4), (4, 1)])

# 查看图
print(DG.nodes()) # 查看节点
print(DG.edges()) # 查看有向边
```

简单分析

1. 图的入度和出度

对于有向图，每个节点都有入度（指向该节点的边的数量）和出度（从该节点出发的边的数量）。

```
print(DG.in_degree()) # 查看所有节点的入度
print(DG.out_degree()) # 查看所有节点的出度

for node, in_deg, out_deg in zip(DG.nodes(), DG.in_degree(), DG.out_degree()):
    print(f"Node {node} has in-degree {in_deg} and out-degree {out_deg}")
```

2. 图的邻接节点

对于有向图，你需要区分前驱节点（指向当前节点的节点）和后继节点（从当前节点出发指向的节点）。

```
print(list(DG.predecessors(2))) # 获取节点2的所有前驱节点
print(list(DG.successors(2)))   # 获取节点2的所有后继节点
```

3. 图的连通性

对于有向图，连通性的概念更加复杂，因为你需要区分强连通（任意两个节点都互相可达）和弱连通（忽略边的方向后任意两个节点都可达）。

```
print(nx.is_strongly_connected(DG)) # 检查图是否是强连通的
print(nx.is_weakly_connected(DG))  # 检查图是否是弱连通的（忽略方向）
```

4. 图的遍历

有向图的遍历同样可以使用深度优先搜索（DFS）和广度优先搜索（BFS），但结果会根据边的方向而变化。

```
# 深度优先搜索
print(list(nx.dfs_preorder_nodes(DG, source=1)))

# 广度优先搜索
print(list(nx.bfs_edges(DG, source=1)))
```

5. 图的绘图

绘图时，`networkx` 会自动根据是否有向来绘制箭头，表示边的方向。

```
import matplotlib.pyplot as plt

# 使用matplotlib绘制有向图
nx.draw(DG, with_labels=True, font_weight='bold', arrowstyle='->', arrowsize=20)
plt.show()
```

通过以上步骤，你可以开始使用 `networkx` 来创建和分析有向图了。`networkx` 的丰富功能允许你进行更深入的图论分析，包括最短路径计算、网络流、社区检测等。

6.2 利用networkx分析图

6.2.1 有哪些指标可以分析图

在复杂网络中，基本指标用于描述和分析网络的结构特性和行为。以下是一些关键的基本指标：

1. 度 (Degree)

- 定义：**节点的度是指与该节点直接相连的边的数量。在无向图中，度即为与该节点相连的边数；在有向图中，分为入度（指向该节点的边数）和出度（从该节点出发的边数）。
- 作用：**度分布 (Degree Distribution) 描述了不同度数的节点在网络中出现的概率或频率，是理解网络拓扑结构和性质的重要工具。

2. 平均路径长度 (Average Path Length)

- 定义：**网络的平均路径长度是指网络中任意两个节点之间距离的平均值，其中距离定义为连接这两个节点的最短路径上的边数。
- 作用：**平均路径长度衡量了网络中节点间的离散程度，反映了网络的整体连接效率。

3. 聚类系数 (Clustering Coefficient)

- 定义：**聚类系数用于描述网络中节点的聚集情况，即网络的紧密程度。对于某个节点，其聚类系数反映了其邻居节点之间实际存在的边数与最大可能边数的比例。
- 作用：**聚类系数是衡量网络局部结构紧密性的重要指标，高聚类系数通常意味着网络中存在许多紧密相连的社区或群组。

4. 度分布 (Degree Distribution)

- 已在上文提及，**此处不再赘述。

5. 模块度 (Modularity)

- 定义：**模块度是衡量网络社区结构强度的一种方法，它评估了网络划分成社区的质量。模块度值越接近1，表示网络划分出的社区结构越强，划分质量越好。
- 作用：**模块度常用于社团划分结果的评估，帮助研究者判断网络是否存在明显的社区结构以及划分是否合理。

6. 加权聚类系数和点强度 (Weighted Clustering Coefficient and Node Strength)

- **加权聚类系数**：对于加权复杂网络，加权聚类系数考虑了边的权重，用于评估节点邻居之间的紧密程度。
- **点强度**：在无向图中，点强度是节点所有连接的边的权重之和；在有向图中，点强度可进一步区分为入强度和出强度。点强度反映了节点在网络中的影响力或重要性。

7. 其他指标

- **介数中心性 (Betweenness Centrality)**：衡量节点在网络中作为“桥梁”的重要性，即经过该节点的最短路径数量。
- **接近中心性 (Closeness Centrality)**：评估节点到网络中其他所有节点的平均距离，反映节点在网络中的可达性或接近程度。

综上所述，复杂网络中的基本指标涵盖了度、平均路径长度、聚类系数、度分布、模块度等多个方面，这些指标共同构成了理解和分析复杂网络结构特性的重要工具。

6.2.2 利用networkx计算这些指标

当然可以，以下我将结合 networkx 的代码示例来为你讲述每个指标的计算方法。

1. 节点度 (Degree)

```
import networkx as nx

# 创建一个图
G = nx.Graph()

# 添加边
G.add_edges_from([(1, 2), (1, 3), (2, 3), (3, 4), (4, 5)])

# 计算所有节点的度
degree_dict = G.degree()
print("所有节点的度:", degree_dict)

# 计算指定节点的度
degree_of_node_1 = G.degree(1)
print("节点1的度:", degree_of_node_1)
```

2. 连通度 (Connectivity)

对于连通度，我们主要关注无向图的点连通度和边连通度，但 `networkx` 直接提供了检查连通性的函数，而不是计算具体的连通度值（即移除多少节点/边后图不再连通）。

```
# 检查图是否连通
is_connected = nx.is_connected(G)
print("图是否连通:", is_connected)

# 对于有向图，检查是否强连通
DG = nx.DiGraph(G) # 将无向图转换为有向图作为示例
is_strongly_connected = nx.is_strongly_connected(DG)
print("有向图是否强连通:", is_strongly_connected)
```

注意：这里并没有直接计算连通度的函数，因为这是一个复杂的问题，通常涉及到图的割点或割边。

3. 聚类系数 (Clustering Coefficient)

```
# 计算所有节点的平均聚类系数
avg_clustering = nx.average_clustering(G)
print("平均聚类系数:", avg_clustering)

# 计算特定节点的聚类系数
clustering_of_node_1 = nx.clustering(G, 1)
print("节点1的聚类系数:", clustering_of_node_1)
```

4. 介数中心性 (Betweenness Centrality)

```
# 计算所有节点的介数中心性
betweenness = nx.betweenness_centrality(G)
print("所有节点的介数中心性:", betweenness)

# 如果你只对部分节点感兴趣，可以指定`nodes`参数
# 例如，计算节点1的介数中心性（但实际上这个函数会计算所有节点，然后你可以从中选择）
```


5. 三角形计数 (Triangle Count)

```
# 计算图中所有三角形的数量
triangles = nx.triangles(G)
total_triangles = sum(triangles.values()) # 因为triangles返回一个字典，键为节点，值为该节点参与的
print("总三角形数量:", total_triangles)

# 或者，如果你想知道特定节点参与的三角形数量
triangles_of_node_1 = triangles[1]
print("节点1参与的三角形数量:", triangles_of_node_1)
```

6. 传递性 (Transitivity)

传递性通常通过聚类系数来间接衡量，但 `networkx` 也提供了直接计算传递性的函数。

```
# 计算图的传递性
transitivity = nx.transitivity(G)
print("图的传递性:", transitivity)
```

注意

- 这些示例假设你已经有一个 `networkx` 图 `G`。在实际应用中，你可能需要根据自己的数据来创建或加载图。
- 对于连通度和其他更复杂的图论问题，如计算最小割集（即移除最少的边或节点使图不连通），`networkx` 可能不直接提供函数，但你可以使用其提供的算法和工具来自己实现这些计算。
- 介数中心性的计算可能非常耗时，特别是对于大型图。如果你只需要部分节点的介数中心性，可以考虑使用 `nodes` 参数来限制计算范围。

6.3 利用networkx求解图的一些基本问题

6.3.1 遍历问题

在 `networkx` 中，你可以使用内置的遍历函数来进行深度优先遍历 (DFS) 和广度优先遍历 (BFS)。这些遍历函数允许你按照特定的顺序访问图中的节点。

深度优先遍历 (DFS)

对于深度优先遍历，`networkx` 提供了 `dfs_edges` 和 `dfs_preorder_nodes` 等函数。但如果你想要一个更直观的DFS遍历，你可能需要自己实现或使用 `dfs_preorder_nodes` 来获取节点的访问顺序。

下面是一个使用 `dfs_preorder_nodes` 进行深度优先遍历的示例：

```
import networkx as nx

# 创建一个图
G = nx.Graph()
# 添加边
edges = [(1, 2), (1, 3), (2, 4), (3, 4), (4, 5)]
G.add_edges_from(edges)

# 深度优先遍历（使用dfs_preorder_nodes）
# 注意：你需要指定一个起始节点
start_node = 1
visited_nodes = list(nx.dfs_preorder_nodes(G, start_node))
print("DFS遍历顺序（从节点{}开始）:".format(start_node), visited_nodes)
```

广度优先遍历 (BFS)

对于广度优先遍历，`networkx` 提供了 `bfs_edges` 和 `bfs_tree` 等函数，但最常用的是 `bfs_edges`，因为它直接给出了遍历的顺序。然而，如果你想要节点的访问顺序而不是边，你可以使用 `bfs_predecessors` 或 `bfs_successors`，或者简单地通过遍历 `bfs_edges` 来构建节点列表。

下面是一个使用 `bfs_edges` 进行广度优先遍历并构建节点访问顺序的示例：

```

import networkx as nx

# 创建一个图
G = nx.Graph()
# 添加边
edges = [(1, 2), (1, 3), (2, 4), (3, 4), (4, 5)]
G.add_edges_from(edges)

# 广度优先遍历
# 注意：你同样需要指定一个起始节点
start_node = 1
visited_nodes = []
for u, v in nx.bfs_edges(G, start_node):
    if u not in visited_nodes:
        visited_nodes.append(u)
    if v not in visited_nodes:
        visited_nodes.append(v)

# 注意：上面的代码可能不是最优的，因为它在添加节点时没有考虑到边的方向（尽管在无向图中这没关系）
# 更简洁的方法是使用集合来自动去重
visited_nodes_set = set()
for u, v in nx.bfs_edges(G, start_node):
    visited_nodes_set.add(u)
    visited_nodes_set.add(v)
visited_nodes = list(visited_nodes_set)

print("BFS遍历顺序（从节点{}开始）:".format(start_node), visited_nodes)

```

然而，`bfs_edges` 生成的遍历顺序实际上是按照边的顺序来的，因此如果你只关心节点的访问顺序而不关心它们是通过哪条边被访问的，你可能想要稍微调整上面的代码，以确保节点只被添加一次到 `visited_nodes` 列表中。上面的 `visited_nodes_set` 示例已经展示了这一点。

另外，`networkx` 的 `bfs_predecessors` 和 `bfs_successors` 函数可以提供从起始节点到每个可达节点的最短路径上的前驱和后继节点信息，这也可以用于构建BFS遍历的节点顺序，但通常需要额外的处理来得到完整的节点列表。

6.3.2 最小生成树问题

在 `networkx` 中，解决最小生成树（Minimum Spanning Tree, MST）问题的一个常用算法是Kruskal算法或Prim算法。`networkx` 提供了 `minimum_spanning_tree` 函数，该函数默认使用Kruskal算法来找到图的最小生成树。

下面是一个使用 networkx 解决最小生成树问题的代码案例：

```
import networkx as nx
import matplotlib.pyplot as plt

# 创建一个加权图
G = nx.Graph()
edges = [
    ('A', 'B', 10),
    ('A', 'C', 6),
    ('A', 'D', 5),
    ('B', 'C', 5),
    ('B', 'D', 15),
    ('B', 'E', 4),
    ('C', 'D', 15),
    ('C', 'F', 12),
    ('D', 'E', 20),
    ('E', 'F', 9)
]

# 添加带权重的边
G.add_weighted_edges_from(edges)

# 找到并绘制最小生成树
T = nx.minimum_spanning_tree(G)

# 绘制原图
plt.figure(figsize=(8, 6))
nx.draw(G, with_labels=True, node_color='lightblue', alpha=0.5, edge_color='gray')
plt.title("Original Graph")
plt.show()

# 绘制最小生成树
plt.figure(figsize=(8, 6))
nx.draw(T, with_labels=True, node_color='lightgreen', alpha=0.5, edge_colors='red', width=2)
plt.title("Minimum Spanning Tree")
plt.show()

# 打印最小生成树的边
print("Edges in the Minimum Spanning Tree:")
for u, v, weight in T.edges(data='weight'):
    print(f"{u} -- {v} weighted {weight}")
```

在这个例子中，我们首先创建了一个加权图 G ，然后使用 `nx.minimum_spanning_tree` 函数找到了这个图的最小生成树 T 。之后，我们使用 `matplotlib` 和 `networkx` 的绘图功能来分别展示原图和最小生成树。最后，我们还打印了最小生成树中所有边的信息。

注意，由于 `networkx` 的 `draw` 函数默认使用的是圆形布局，这可能导致在某些情况下，图的绘制看起来不是很清晰或直观。你可以通过调整 `draw` 函数的参数（如 `pos` 参数，它允许你自定义节点的位置）来改善这一点。然而，在这个简单的示例中，我们使用了默认的圆形布局来展示结果。

另外，`networkx` 的 `minimum_spanning_tree` 函数默认使用Kruskal算法，因为它适用于稀疏图，并且不需要事先知道图的所有顶点。如果你需要更详细的控制，或者想要使用Prim算法（它更适用于密集图），你可能需要自己实现这些算法，因为 `networkx` 的标准库中并没有直接提供Prim算法的实现作为 `minimum_spanning_tree` 的选项。不过，对于大多数实际应用来说，Kruskal算法已经足够好了。

6.3.3 最短路径问题

在 `networkx` 中，解决最短路径问题通常涉及到使用几种不同的算法，具体取决于问题的具体要求和图的特性（如有向图、无向图、是否包含负权边等）。以下是一些常用的算法和它们在 `networkx` 中的实现方式：

1. Dijkstra算法

Dijkstra算法是一种用于解决单源最短路径问题的贪心算法。它适用于带有非负权重的图。在 `networkx` 中，可以使用 `dijkstra_path` 函数来计算从源节点到目标节点的最短路径，以及使用 `dijkstra_path_length` 函数来计算最短路径的长度（即路径中所有边的权重之和）。

示例代码：

```
import networkx as nx

# 创建一个有向图
G = nx.DiGraph()
# 添加带权重的边
G.add_edge('A', 'B', weight=3)
G.add_edge('A', 'C', weight=2)
G.add_edge('B', 'C', weight=1)
G.add_edge('B', 'D', weight=4)
G.add_edge('C', 'D', weight=2)
G.add_edge('C', 'E', weight=1)
G.add_edge('D', 'E', weight=3)

# 计算从节点'A'到节点'E'的最短路径
shortest_path = nx.dijkstra_path(G, 'A', 'E', weight='weight')
print("最短路径:", shortest_path)

# 计算从节点'A'到节点'E'的最短路径长度
shortest_path_length = nx.dijkstra_path_length(G, 'A', 'E', weight='weight')
print("最短路径长度:", shortest_path_length)
```

2. Bellman-Ford算法

Bellman-Ford算法也是一种用于解决单源最短路径问题的算法，但它能够处理带有负权边的图。不过，它的时间复杂度较高，为 $O(VE)$ ，其中 V 是顶点数， E 是边数。在 `networkx` 中，可以使用 `bellman_ford_path` 和 `bellman_ford_path_length` 函数来计算最短路径和最短路径长度。

注意：由于Bellman-Ford算法的特性，它通常用于需要处理负权边的场景。

3. Floyd-Warshall算法

Floyd-Warshall算法是一种用于解决所有顶点对之间最短路径问题的动态规划算法。它能够处理带有正权、负权（但不能有负权回路）的图。在 `networkx` 中，可以使用 `floyd_warshall` 函数来计算图中所有顶点对之间的最短路径长度，但它不直接返回路径本身。如果需要路径，可以通过额外的步骤从最短路径长度矩阵中重建路径。

示例代码（仅计算最短路径长度）：

```
import networkx as nx

# 创建一个有向图
G = nx.DiGraph()
# 添加带权重的边
G.add_edge('A', 'B', weight=3)
# ... (省略其他边的添加)

# 计算所有顶点对之间的最短路径长度
path_lengths = nx.floyd_warshall_numpy(G, weight='weight')
print("最短路径长度矩阵:\n", path_lengths)

# 注意: Floyd-Warshall算法不直接返回路径, 但可以从path_lengths中重建路径
```

- 对于单源最短路径问题, 如果不存在负权边, 推荐使用Dijkstra算法 (`dijkstra_path` 和 `dijkstra_path_length`) 。
- 如果需要处理负权边, 则应使用Bellman-Ford算法 (`bellman_ford_path` 和 `bellman_ford_path_length`) 。
- 对于需要求解所有顶点对之间最短路径长度的问题, 可以使用Floyd-Warshall算法 (`floyd_warshall_numpy`) , 但请注意它不直接返回路径。

在实际应用中, 应根据问题的具体要求和图的特性选择合适的算法。

扩展阅读

如果你对图论当中一些基本问题有了基本了解, 想要深入学习复杂网络理论及其在深度学习中的应用——图神经网络这一模块, 那么我推荐你来看一看葡萄书:

[datawhalechina/grape-book](https://datawhalechina.github.io/grape-book): 图深度学习 (葡萄书) , 在线阅读地址:
<https://datawhalechina.github.io/grape-book>)